

Claude Code CLI — EntraID Auth via Default Azure CLI Login

Configures the Claude Code CLI to authenticate to this self-hosted AIGW gateway using whatever token `az account get-access-token` already hands back from a normal `az login` — no dedicated EntraID app registration. This is the simpler counterpart to [claude-code-cli-entra-auth-dedicate-app.md](#).

How it works, and why it's simpler

This approach leverages the existing JWT from the account that is received by running:

```
az account get-access-token --query 'accessToken' -o tsv
```

It returns a standard Azure Resource Manager token (audience <https://management.azure.com/>) for whatever account is already logged in. That token still works with the gateway because of how JWT local auth resolves identity — see [jwt-auth-resolution-logic.md](#):

1. The gateway cryptographically verifies the token against the org's JWKS (any valid Microsoft-signed token for the right tenant passes this — the audience doesn't need to match a custom API).
2. It reads an email-shaped claim off the token (`email_id` → `email` → `preferred_username` → `upn`) — a standard ARM token from a normal Entra login already carries `upn/preferred_username`.
3. It looks that email up against AIRS Gateway and resolves the workspace from the **user's actual workspace membership**, overriding any `x-portkey-workspace` header or deployment default.

No custom claims are needed because the gateway doesn't rely on the token to *declare* the org/workspace — it verifies the token, extracts an email, and looks up membership itself that is allocated via SCIM. This only works because `ORGANISATIONS_TO_SYNC` in this deployment resolves to a single org (so `portkey_oid` isn't required either).

```
Claude Code CLI
├── apiKeyHelper: az account get-access-token --query 'accessToken' -o
tsv
│   (plain ARM token, default tenant/account, no custom resource)
├── sends requests to AIGW gateway (http://127.0.0.1:8787)
│   Authorization: Bearer <token>
│   │   gateway verifies signature via org JWKS, extracts email
claim,
│       looks up Portkey workspace membership by that email
```

When to use this vs. the dedicated-app approach

	This doc (default az login)	Dedicated app
EntraID app registration	Not required	Required
Claims Mapping Policy	Not required	Required
ENTRAID_CLIENT_ID / ENTRAID_TENANT_ID	Not needed	Required
Works with multiple orgs on one deployment	No — needs a single-org ORGANISATIONS_TO_SYNC	Yes — portkey_oid claim picks the org
Workspace resolution	By Portkey membership lookup on the user's email	Explicit portkey_workspace claim (or membership fallback)
Setup effort	Minimal — just az login	Higher — app registration, policy, pre-authorization

Use this approach for a single-org deployment where you'd rather skip EntraID app registration entirely. Use the dedicated-app approach for multi-org gateways or when you need explicit control over workspace assignment (e.g. via extension attributes).

Prerequisites

- The gateway stack running, with JWT_ENABLED: ON (already set — see docker-compose.yml)
- This deployment's ORGANISATIONS_TO_SYNC set to a **single** org UUID
- Your Entra account's email registered as a member of a workspace in that AIGW org via SCIM
- Azure CLI installed (brew install azure-cli) and logged in (az login) to the correct tenant

1. Configure Claude Code CLI

Edit ~/.claude/settings.json:

```
{
  "apiKeyHelper": "az account get-access-token --query 'accessToken' -o tsv",
  "env": {
    "ANTHROPIC_BASE_URL": "http://127.0.0.1:8787",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "@vertex/anthropic.claude-sonnet-5[1m]",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "@vertex/anthropic.claude-opus-5",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "@vertex/anthropic.claude-sonnet-5",
    "CLAUDE_CODE_API_KEY_HELPER_TTL_MS": "3000000"
  }
}
```

Field	Value
apiKeyHelper	Inline command — no separate script needed. Runs az account get-access-token with no --resource/-tenant, using whatever account/tenant is already active in the local Azure CLI session

Field	Value
<code>ANTHROPIC_BASE_URL</code>	Gateway URL — <code>http://127.0.0.1:8787</code> for local Docker
<code>ANTHROPIC_DEFAULT_SONNET_MODEL</code> / <code>_OPUS_MODEL</code> / <code>_HAIKU_MODEL</code>	Model IDs routed through the gateway's <code>@vertex</code> integration — note the <code>@vertex/</code> prefix (vs. the bare <code>anthropic.</code> prefix used when the model resolves through a Portkey config) and the optional <code>[1m]</code> suffix for the 1M-context Sonnet variant
<code>CLAUDE_CODE_API_KEY_HELPER_TTL_MS</code>	<code>3000000</code> (~50 min) caches the token between calls, matching typical ARM token lifetime

Everything else in `settings.json` (`permissions`, `model`, `modelPicker`, `effortLevel`, `theme`, etc.) is unrelated to the gateway auth and can be set to your own preferences.

No `x-portkey-config` header set here. Because the org/workspace is resolved from the token itself (not a config lookup), routing relies on the workspace's default config in the AIGW control plane. Set one there if requests fail with no route.

2. Verify with curl

```
TOKEN=$(az account get-access-token --query 'accessToken' -o tsv) && \
curl http://127.0.0.1:8787/v1/messages \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"model": "@vertex/anthropic.claude-sonnet-5[1m]", "max_tokens": 250,
"messages": [{"role": "user", "content": "hi"}]}'
```

Decode the token to confirm it carries an email-shaped claim (`upn`, `preferred_username`, or `email`) — that's the only thing the gateway needs from it:

```
echo $TOKEN | cut -d. -f2 | base64 -d 2>/dev/null | python3 -m json.tool
```

3. Start Claude Code

```
claude
```

If the local Azure CLI session has expired, run `az login` yourself first — this approach does not trigger an interactive login automatically the way `get-az-token.sh` does (there's no wrapper script to catch the empty-token case).

Troubleshooting

Symptom	Likely cause	Fix
401 / JWT verification failed	Not logged in, or logged into the wrong tenant	<code>az login --tenant <tenant-id></code>
403 "Organisation ID not found in token ... and no single ORGANISATIONS_TO_SYNC configured"	Deployment has more than one org in ORGANISATIONS_TO_SYNC	Use the dedicated-app approach instead, or split into single-org deployments
403 "No workspace could be resolved for this token"	Account's email isn't a member of any workspace in the Portkey org	Add the user to a workspace in Portkey, or check <code>upn/email</code> claim matches their Portkey membership email exactly
403 "User is not a member of the resolved workspace <slug>"	Workspace default resolved to one the user isn't in	Add the user to that workspace, or unset the org's <code>default_workspace</code> if it's the wrong one
Authenticated as WORKSPACE_SERVICE instead of the expected user	Token has no email-shaped claim (e.g. logged in via a service principal / managed identity)	Log in with <code>az login</code> as a real user account, not a service principal
messages is not supported by vertex-ai	Model name missing the routing prefix	Use <code>@vertex/anthropic.claude-sonnet-5[1m]</code> (or the equivalent for the model you're routing)
Stale token after re-login	CLAUDE_CODE_API_KEY_HELPER_TTL_MS cache hasn't expired	Wait out the TTL, or restart <code>claude</code>
az: command not found	Azure CLI not installed	<code>brew install azure-cli</code>

Decode the token to inspect claims:

```
echo $TOKEN | cut -d. -f2 | base64 -d 2>/dev/null | python3 -m json.tool
```